

DispatchDesk: A Dispatch Copilot That Helps Dark Store Managers Keep the 10-Minute Promise the Right Way

Document type: Six-page narrative memo **Team:** DispatchDesk (5 members across RAG and six-pager; Setup and integration) **Status:** Draft for team review **Date:** 27 September 2026; revised 2 October 2026

Reading note: this memo is meant to be read silently, start to finish, before discussion. The main body ends at Section 9. Appendices hold supporting data and are not required reading.

Data note: The data used in this memo is synthetically generated for demonstration purposes and does not represent real store operations.

1. Introduction

A quick-commerce dark store makes one promise to every customer: groceries at the door in about ten minutes. When that promise holds, nobody notices the store manager. When it breaks, usually between seven and ten in the evening or the moment it starts raining, the manager is the one person expected to fix it in real time, with a phone ringing, a dispatch screen full of red, and regional operations watching an hourly SLA number.

DispatchDesk is a proposed last-mile dispatch copilot for that manager. It is designed to answer two questions: *why* are deliveries slipping right now, and *what should I do first?* It answers them in plain language, grounded in the store's actual order queue, rider fleet, and delivery-stage history, and in the store's own dispatch rules and rider safety policy. It remembers each store's operating preferences across shifts. Just as importantly, it has firm limits: it never invents a queue count or ETA, never pushes riders toward unsafe or over-hours work to rescue an SLA, and never takes a dispatch action on its own. Every assignment, batch, incentive, or customer message it suggests is a draft that the manager approves or rejects.

This memo seeks agreement on the scope, success criteria, and four-week plan. We will evaluate whether DispatchDesk helps users identify the bottleneck and choose a policy-compliant action faster.

2. The Problem

Where the ten minutes go

Every quick-commerce delivery passes through three stages, and a delay almost always lives in one of them. **Pick-pack** is the time from order placed to packed and ready, with a target of about three minutes. **Rider wait** is the time a packed order sits on the shelf before a rider picks it up, with a target of about one and a half minutes. **Ride** is pickup to doorstep, with a target of about five minutes within a two-kilometre radius. Average delivery time is simply the sum of the three.

The SLA percentage that managers are measured on hides this structure. It also behaves in a way that surprises people: it falls off a cliff rather than a slope. Our operations playbook notes that when average delivery time drifts from roughly eight minutes to twelve, the share of orders inside ten minutes can collapse from around ninety percent to under fifty. A manager who only sees the percentage sees a disaster; he does not see which stage caused it, and so he cannot tell whether the fix is more riders, more pickers, a smaller delivery radius, or simply an honest ETA.

Two evenings, in numbers

Our dataset for store DS-BLR-014 in Koramangala contains two consecutive evenings that illustrate the problem precisely. On 23 September, a dry evening, the 8 pm hour handled 88 orders with 20 riders online. Pick-pack averaged 2.8 minutes, rider wait 1.1, and ride 4.6, for an average delivery time of 8.5 minutes and an SLA of 87 percent. The following evening, 24 September, it rained. The 8 pm hour saw 96 orders, but only 17 riders were online. Pick-pack rose modestly to 3.3 minutes, ride rose to 6.0, and rider wait jumped to 3.6 minutes. Average delivery time reached 12.9 minutes and the SLA fell to 44 percent. At 9 pm it was worse: 102 orders, 15 riders, rider wait of 4.4 minutes, and an SLA of 31 percent.

Broken down this way, the story is clear. Of the 4.4-minute increase in delivery time at 8 pm, 2.5 minutes came from rider wait, 1.4 from slower rides on wet roads, and only 0.5 from the store floor. More orders arrived while fewer riders were available, which is the classic rain pattern described in our playbook. The right response is to add rider capacity, relieve rider shortage through careful batching, and reset customer expectations, not to push pickers harder and certainly not to push riders faster. Yet a manager looking at a single number, "SLA 44%", has no way to reach that conclusion in the thirty seconds he has.

The same pattern, live

The snapshot at 20:14 on 25 September shows how this feels in the moment. Six packed orders are waiting for a rider, and only two riders are free. That is three pending orders per available rider, above the playbook's warning threshold of two, beyond which waits climb quickly. Two more riders are returning within three minutes, one rider is on a break, one has gone offline because of weather, and a standby rider could arrive in twenty minutes. The oldest waiting order is eight minutes old, the point at which the recorded demo policy calls for a draft delay notice. One of the waiting orders contains frozen items and cannot be batched. One of the returning riders has already been on shift for 8.5 hours, half an hour short of the nine-hour maximum. Every one of those facts matters to the next decision, and none of them is visible from the SLA percentage.

The pressure that makes shortcuts tempting

The deeper problem is not information alone; it is what pressure does to decisions. When regional operations is watching the SLA hour by hour, the tempting shortcuts are unsafe ones: tell riders to "make the ten minutes no matter what", dock pay for late orders, keep a tired rider on through the peak, or batch orders that should not be combined. Last quarter, an aggressive "hit the SLA at any cost" push at our persona's store ended in a rider near-miss and a labor complaint. Our playbook is unambiguous that riders must never be pressured or incentivized to speed or break traffic rules, must never be penalized for weather- or safety-driven delays, and must never work past nine hours on shift or four hours without a fifteen-minute break. A tool that helps a manager hit the SLA by quietly ignoring those rules would make the real problem worse.

3. The Customer

Our customer persona is Karthik Reddy, a 29-year-old dark store manager in Koramangala, Bengaluru. His store handles roughly 600 orders a day on a ten-minute promise, stocks about 2,500 SKUs, and runs with 9 pickers and around 20 riders per shift, a mix of employed and gig workers. For most of the day the store runs itself. Between seven and ten in the evening, or the moment it rains, it does not.

In those windows Karthik makes dozens of split-second calls: which order goes out first, whether two orders can share one bike, when to call in the standby rider, when to shrink the delivery radius. He is competent and he knows his riders, but his dashboards describe *what* is happening rather than *why*,

so every call rests on instinct. He wants to ask questions the way he would ask an experienced colleague: "What's going wrong right now?" or "Why did last night fall apart?" He wants answers built from his own queue and rider data, not generic advice.

Karthik is also wary, and his wariness is the most important thing we know about him. After last quarter's near-miss, he does not want an assistant that helps him hit the number at the riders' expense, and he does not want a bot that takes control of dispatch. He wants something that helps him hit the promise the right way, and that tells him clearly, with reasons, when a shortcut is unsafe or against policy. His objective is to keep deliveries fast and the fleet safe during peaks, without becoming the manager who burns out riders and without handing dispatch to software.

4. Tenets

These are the principles we will use to settle disagreements during the build. They are listed in priority order; when two conflict, the earlier one wins.

1. **Rider safety comes before the SLA.** DispatchDesk will never suggest, draft, or endorse anything that pressures riders to speed, break traffic rules, work past their hour limits, or skip mandatory breaks, and it will never recommend penalizing riders for weather- or safety-driven delays. When asked to, it says no, explains why using the store's own policy, and offers compliant alternatives.
2. **The manager decides; DispatchDesk proposes.** Anything that changes the state of the world, whether assigning a rider, batching orders, activating an incentive, changing the radius, calling in a rider, or messaging a customer, is produced as a draft for explicit approval. Nothing executes on its own.
3. **No number without a source.** Every queue count, delivery time, SLA percentage, rider status, and ETA must come from a tool call, and live data must carry its "as of" time. ETAs are ranges derived from data, labelled as estimates, and never phrased as promises. If data is unavailable or stale, DispatchDesk says so plainly.
4. **The store's preferences are respected, not overridden.** When a manager sets an alert threshold, a batching constraint, or an incentive cap, DispatchDesk applies it in every later shift. If a suggestion would conflict with a stored preference, it surfaces the conflict instead of silently choosing.
5. **Explain the why, then the what.** A diagnosis names the stage that is causing the delay and the evidence for it before it recommends an action, so the manager can judge the advice rather than simply follow it.

5. The Solution

How DispatchDesk works

DispatchDesk is a conversational assistant with three distinct sources of truth, and keeping them separate is the core of the design. **Facts** come only from tools: a live dispatch-status tool returns the current order queue and rider states with an "as of" timestamp, and a delivery-metrics tool returns hourly stage breakdowns for periods available in the dataset. **Rules** come only from retrieval over the store's operating documents: dispatch procedures, the delay root-cause guide, batching and cold-chain rules, the rain and surge playbook, customer-communication guidelines, and the rider safety and working-hours policy. **Preferences** come only from memory: the alert thresholds, batching constraints, and incentive caps each manager has set. The language model's job is to reason across these three sources and write a clear answer. It is never the source of a number or a rule.

Deterministic checks are done in code, not by the model. Whether pending orders per available rider exceeds a threshold, whether a pair of orders meets every batching condition, and whether a rider has reached the nine-hour or four-hour limit are all computed exactly from tool data. The model explains the results. A guardrail layer checks every response before it reaches the manager, confirming that figures trace to tool output, that proposed actions are framed as drafts, and that nothing conflicts with safety policy or stored preferences.

A worked example

At 20:14 on 25 September, Karthik types: "Orders are backing up right now, what's going on and what should I do first?" DispatchDesk calls the live dispatch tool and retrieves the delay-diagnosis and batching guidance. It replies that, as of 20:14, six packed orders are waiting on two available riders, three pending orders per available rider, so rider wait is the bottleneck rather than the store floor. It then proposes, in order, the playbook's first actions: dispatch the oldest waiting order first; expect two riders back within three minutes; call in the standby rider, who is about twenty minutes away; and assess the two Zone B orders containing nine items as batching candidates. They cannot be recommended as a batch until the missing detour data and every other eligibility check are verified. It notes that the order containing frozen items must go out as a single drop, that the oldest order has reached the eight-minute limit, is ineligible for batching, and qualifies for a draft delay notice, and that the returning rider at 8.5 hours on shift can only be dispatched within the 9-hour limit, verified from tool data, and must not be scheduled into the peak. Each proposed action appears as a draft with an approve button. Nothing moves until Karthik approves it and current eligibility is rechecked.

Looking back as well as forward

The same approach answers retrospective questions. Asked why the SLA fell on the evening of 24 September compared with the night before, DispatchDesk pulls metrics for both periods, compares the three stages hour by hour, notes the rain flag and the drop from 20 riders online to 15 while orders rose, and concludes that rider wait was the dominant increase, citing the actual figures and the rain playbook's recommended levers for next time.

Saying no, usefully

When a request crosses a line, DispatchDesk refuses clearly and stays helpful. If asked to tell riders to "make the ten minutes no matter what" and dock pay for late orders, it explains that both violate the rider safety policy and offers the compliant levers instead: an honest, longer ETA, a temporarily smaller serviceability radius, the pre-approved surge incentive within the store's cap, policy-compliant batching, and a standby call-in. If asked to keep a rider on through the peak when the rider-status tool shows he is at 8.5 hours, it declines to schedule him past the nine-hour limit and suggests the standby rider, zone rebalancing, a smaller radius, or a request to regional operations for extra capacity.

Memory across shifts

A manager can tell DispatchDesk, "On weekends after 7 pm, alert me when pending orders per available rider goes above 2, and never batch frozen items with anything else." DispatchDesk stores the threshold, the window it applies to, and the batching constraint, confirms them, and applies them automatically in later shifts without being reminded. If a later suggestion would break one of them, it flags the conflict for the manager to resolve. Preferences cannot relax mandatory policy; alerts are evaluated during interactions.

What the manager sees

The interface is a chat window with an expandable agent trace showing which tools were called, when each piece of live data was captured, which documents were retrieved, and which stored preferences were applied. Visible badges show data freshness, guardrail status (for example, when a request was refused or reframed), and cache hits. An observability dashboard tracks tool-call failures, stale-data events, and guardrail triggers. When live data cannot be reached, DispatchDesk shows the last snapshot with its time. Actions requiring current state wait for refreshed data.

6. Goals and Non-Goals

Within four weeks, our goal is a working demonstration that answers all six sample manager queries in our requirements correctly and safely. That means diagnosing live backlogs and past-evening slowdowns with figures from the dataset; proposing policy-compliant batches and explaining every excluded order; giving ETAs as data-derived, labelled ranges; refusing unsafe rider pressure and over-hours scheduling with compliant alternatives; remembering store preferences across at least two separate sessions; keeping every action in draft until approved; and exposing tool failures, stale data, and graceful degradation through observability. We also aim to measure improvement with an automated evaluation suite, run before and after a round of error-analysis fixes.

Several things are deliberately out of scope. It will not optimize routes or compute precise travel times; ride estimates come from historical zone averages. It will not execute any action for real; approval simulates execution in the demo. It will not forecast demand or make predictive claims; any "what-if" estimate, if we reach that stretch goal, will be labelled as educational and non-predictive. It will not replace the manager's judgment or regional operations' authority, and it will not grant exceptions to safety or working-hours policy under any circumstances, including incentive amounts above the store's cap, which require regional sign-off.

7. Risks and Mitigations

The assistant invents a number or an ETA. Language models produce plausible figures readily, and a fabricated queue count or promised delivery time would destroy the manager's trust and mislead customers. We will instruct the model never to state figures that were not returned by a tool, compute all derived numbers in code, have the guardrail layer check that figures in a response trace to tool output, and include fabrication checks in the evaluation suite with a target of zero.

The assistant helps with an unsafe shortcut under indirect phrasing. A direct request to make riders speed is easy to catch; "just motivate them" or "he volunteered to stay" is harder. We will ground refusals in the retrieved safety policy, verify rider hours from the tool rather than trusting the request, test the guardrails with indirect phrasings, and harden them against anything that succeeds.

The guardrails over-block. An assistant that refuses benign questions such as "which riders are due a break?" will be abandoned. Our guardrail tests include benign queries, and we will track false refusals as a metric alongside correct refusals.

Live data is stale or unavailable. In a real peak, the dispatch API could time out. Live data will be fetched fresh with only a short-lived cache, always shown with its "as of" time, and, when unreachable, replaced by the last snapshot with an explicit warning. We will simulate timeouts to test this.

The manager over-trusts the assistant. A fluent answer can feel more certain than the data supports. Diagnoses will show their evidence, ETAs will be ranges labelled as estimates, the trace panel will show sources, and every action will require approval.

Retrieval misses the right rule. If the batching or safety section is not retrieved, the answer will be ungrounded. Our corpus is small, so we will chunk by section, test that the right sections appear in the top three results for each sample query, and track retrieval misses in error analysis.

Third-party cost, latency, and availability. DispatchDesk depends on external language-model and embedding services. API usage can raise operating costs, while slow responses or outages can delay or prevent answers. We will track cost per answer, provider latency, and failures; reuse stored embeddings and cache stable content; and set timeouts with limited retries. If a provider remains unavailable, the assistant will report the limitation clearly.

8. Success Metrics

We will judge the demonstration by measurable outcomes produced by our own evaluation harness and dashboard, not by impressions. The minimum acceptance metric is the pass rate on the six sample queries, with a target of six out of six after error-analysis fixes, and a recorded improvement over the baseline run. Alongside it, we will track fabricated figures in responses, with a target of zero; refusal accuracy on unsafe requests, targeting every unsafe test prompt refused with compliant alternatives offered; false refusals on a set of benign dispatch questions, targeting zero; and preference-recall accuracy across sessions, targeting correct unprompted recall of every stored preference in the second session.

We will also test unfamiliar scenario variations and compare decision time and correctness with the same records and policies used without the assistant. Faster decisions count as improvement only when correctness is maintained.

Operational metrics round out the picture: retrieval hit rate, meaning the share of test queries whose correct playbook section appears in the top three results; tool-call failure rate and stale-data occurrences, with every stale or failed call handled by a labelled fallback rather than a crash; cache hit rate and the latency improvement on repeated historical-metrics queries; and median end-to-end response time, with a stretch target of under three seconds. All metrics will be visible on the dashboard during the final demo.

9. Plan

We will build DispatchDesk in four one-week phases, each ending in a demo. In **Week 1**, we lay foundations: this memo and a PR/FAQ, the repository, a system prompt encoding our tenets, the completed synthetic dataset, the retrieval corpus and pipeline, and a Gradio chat interface that answers "why are deliveries slipping?" from the playbook. In **Week 2**, we add the live dispatch-status and delivery-metrics tools exposed through MCP, a memory schema for store preferences, recall across two sessions, and the agent trace panel. In **Week 3**, we codify and implement guardrails, test refusals of unsafe rider pressure and over-hours scheduling alongside benign queries, add caching with a short time-to-live for live data, run all six sample queries end to end, and add freshness, guardrail, and cache badges. In **Week 4**, we add end-to-end tracing and a dashboard, build and run the evaluation suite, perform error analysis and apply the top fixes, handle edge cases such as API timeouts and ambiguous references, and rehearse and record the demo.

Appendix A: Evening Comparison, Store DS-BLR-014

Source: synthetic [dispatch dataset](#), HourlyMetrics sheet. Times in minutes.

Date	Hour	Orders	Pick-pack	Rider wait	Ride	Avg delivery	SLA %	Riders online	Rain
23 Sep	20	88	2.8	1.1	4.6	8.5	87	20	No
23 Sep	21	84	2.8	1.2	4.6	8.6	86	20	No
23 Sep	22	55	2.5	0.8	4.4	7.7	91	16	No
24 Sep	20	96	3.3	3.6	6.0	12.9	44	17	Yes
24 Sep	21	102	3.5	4.4	6.4	14.3	31	15	Yes
24 Sep	22	68	3.0	2.5	5.6	11.1	58	15	Yes

Change in average delivery time, rain evening versus dry evening: at 20:00, +4.4 minutes (rider wait +2.5, ride +1.4, pick-pack +0.5); at 21:00, +5.7 minutes (rider wait +3.2, ride +1.8, pick-pack +0.7). Rider wait accounts for roughly 56 to 57 percent of the increase in both hours.

Appendix B: Live Snapshot Summary, 25 September 2026, 20:14

Source: original synthetic workbook, LiveOrders and RiderStatus sheets. Riders are referred to by ID; current application scenarios have their own records and dates.

The queue holds 12 orders: 6 packed and waiting for a rider, 4 being picked, and 2 out for delivery. The oldest waiting order, ORD-70411, has waited 480 seconds. ORD-70413 contains frozen items. Of 9 riders, 2 are available (RDR-103, RDR-104), 2 are on delivery, 2 are returning within 3 minutes (RDR-105, RDR-107), 1 is on break, 1 is offline because of weather, and 1 is on standby about 20 minutes away (RDR-109). RDR-105 has been on shift for 8.5 hours. Pending orders per available rider: $6 \div 2 = 3.0$, above the playbook threshold of 2.

Appendix C: Sample Queries and Expected Behavior

#	Manager query (abridged)	What DispatchDesk must do
1	Orders backing up, what do I do first?	Live-status diagnosis plus prioritized actions as drafts
2	Why did SLA fall last night vs. the night before?	Stage-by-stage comparison from metrics, rider wait as the largest increase
3	Raining, can I batch, what ETA should I show?	Eligible batches, excluded orders with reasons, ETA range labelled as estimate
4	Make riders hit 10 minutes no matter what, dock pay	Refuse, cite safety policy, offer compliant levers
5	Remember my weekend alert threshold and frozen-batching rule	Store, confirm, recall in a later session, flag conflicts
6	Keep the rider at 8.5 hours on till close	Verify hours via tool, decline past limits, offer alternatives

Appendix D: Key Policy Thresholds (from the Dispatch Playbook)

Includes the team's [recorded demo policy decisions](#).

Rule	Threshold
Stage targets	Pick-pack about 3 min; rider wait about 1.5 min; ride about 5 min within 2 km
Rider-wait warning	Above about 2 pending orders per available rider
Batch size	At most 2 orders per rider (3 only with manager approval in a declared surge)
Batch geography	Same or adjacent zones; added detour must satisfy both caps: about 0.8 km and 2 min
Batch items	15 or fewer combined
Cold chain	Frozen or ice-cream items are single-drop only
Batch age limit	Not allowed at or beyond 8 min in queue
Proactive delay notice	Draft prepared at 8 min or later
Maximum shift	9 hours including breaks
Mandatory break	At least 15 min after every 4 hours without a break